MASTER PDF

Intratrade Complete Documentation Pack

This merged PDF combines all generated project, deployment, client-facing, and branded trading app documents into one Adobe-friendly deliverable.

Generated on 2026-03-18 | Workspace: F:\Trading | Total documents: 17

Included Documents

1. Trade Output Summary
2. AWS README
3. AWS Deployment Guide
4. Docker Kubernetes Dhan Guide
5. Scheduler Guide
6. Trading App Architecture
7. Trading App Deployment
8. Trading App Architecture Diagram
9. Trading App Production Runbook
10. Client Executive Overview
11. Client Architecture
12. Client Deployment
13. Broker Integration Flow
14. Intratrade Master Documentation
15. Intratrade Client Proposal
16. Intratrade Commercial Proposal
17. Intratrade Premium Master Dark

INTRATRADE COMBINED DOCUMENTATION

Trade Output Summary

F:\Trading\data\pdf_html\trade.html

Intratrade Fix Report

Intratrade Dashboard Fix Report

Generated: 2026-03-16 (IST)

Problem
Dashboard summary was misleading:

- `Intratrade alert` showed many rows (e.g., `Trades: 103`) but `Win rate: 0.00%`, `Total PnL: 0.00`, `Last exit: N/A`, `Last reason: N/A`.

Root cause: the dashboard was sending **non-closed rows** (signals / indicator rows / mixed rows) into the **closed-trade** summarizer.

Fix Location
- `F:\Trading\src\telegram_notifier.py` â†' `build_trade_summary()` (around line 12).

What I Changed (Steps)
1. **Classify input rows** before summarizing:
- Indicator rows: contain `market_signal`
- Execution candidates: contain `signal_time` + `side`
- Trade signals (entries without exits): contain `entry_price/stop_loss/target_price` but no `pnl/exit_*`
2. **Only compute Win rate / PnL on closed trades**:
- Filter to rows that contain `pnl` and at least one of `exit_time` / `exit_reason`.
3. **Fallback summaries** for signals:
- If there are no closed trades, show a `Signal alert` with last time/side/entry/SL/target.

How To Export To PDF
Option A (recommended):
1. Open this file: `F:\Trading\intratrade_fix_report.html`
2. Press **Ctrl+P** â†' choose **Save as PDF**.

Option B:
- Open `F:\Trading\intratrade_fix_report.md` and print to PDF from your editor.

INTRATRADE COMBINED DOCUMENTATION

AWS README

F:\Trading\data\pdf_html\README.html

AWS README

AWS Deployment Files

This folder contains ready-to-use deployment files for hosting Intratrade on AWS EC2.

Files
- `deploy/aws/intratrade.service`: systemd service for Streamlit app
- `deploy/aws/chandudevopai.shop.nginx.conf`: nginx reverse proxy config
- `deploy/aws/iam-policy-s3.json`: IAM policy template for S3 access
- `deploy/aws/deploy.sh`: one-shot setup script (Ubuntu)
- `deploy/aws/deploy-amzn-linux-2023.sh`: one-shot setup script (Amazon Linux 2023)
- `deploy/streamlit/config.toml`: streamlit server config for reverse proxy mode

Quick steps (manual)
1. Copy streamlit config:
- `cp deploy/streamlit/config.toml .streamlit/config.toml`
2. Install service:
- `sudo cp deploy/aws/intratrade.service /etc/systemd/system/`
- `sudo systemctl daemon-reload && sudo systemctl enable --now intratrade`
3. Install nginx vhost:
- Ubuntu/Debian:
- `sudo cp deploy/aws/chandudevopai.shop.nginx.conf /etc/nginx/sites-available/chandudevopai.shop.conf`
- `sudo ln -s /etc/nginx/sites-available/chandudevopai.shop.conf /etc/nginx/sites-enabled/`
- Amazon Linux 2023:
- `sudo cp deploy/aws/chandudevopai.shop.nginx.conf /etc/nginx/conf.d/chandudevopai.shop.conf`
- `sudo nginx -t && sudo systemctl reload nginx`
4. Issue SSL cert:
- `sudo certbot --nginx -d chandudevopai.shop -d www.chandudevopai.shop`

One-shot script (Ubuntu)
Update placeholders in `deploy/aws/deploy.sh`:
- `REPLACE_GIT_REPO_URL`
- `REPLACE_EMAIL`

Run:
```bash
chmod +x deploy/aws/deploy.sh
./deploy/aws/deploy.sh
```

```

## One-shot script (Amazon Linux 2023)
Update placeholders in `deploy/aws/deploy-amzn-linux-2023.sh`:
- `REPLACE_GIT_REPO_URL`
- `REPLACE_EMAIL`

Run:
```bash
chmod +x deploy/aws/deploy-amzn-linux-2023.sh
./deploy/aws/deploy-amzn-linux-2023.sh
```
```

INTRATRADE COMBINED DOCUMENTATION

AWS Deployment Guide

F:\Trading\data\pdf_html\DEPLOYMENT_GUIDE.html

AWS Deployment Guide

Deployment Scripts + PDFs

Deploy
- Ubuntu: `deploy/aws/deploy.sh`
- Amazon Linux 2023: `deploy/aws/deploy-amzn-linux-2023.sh`

Undeploy / Rollback
- Ubuntu: `deploy/aws/undeploy.sh`
- Amazon Linux 2023: `deploy/aws/undeploy-amzn-linux-2023.sh`

Nginx
- `deploy/aws/chandudevopai.shop.nginx.conf`

Service
- `deploy/aws/intratrade.service`

Streamlit Config
- `deploy/streamlit/config.toml`

IAM Policy
- `deploy/aws/iam-policy-s3.json`

Notes
1. Update placeholders in deploy scripts before running.
2. Test nginx before reload: `sudo nginx -t`.
3. Use certbot for TLS certs.

INTRATRADE COMBINED DOCUMENTATION

Docker Kubernetes Dhan Guide

F:\Trading\data\pdf_html\DOCKER_KUBERNETES_DHAN_GUIDE.html

Docker Kubernetes Dhan Guide

Docker + Kubernetes + Dhan API (Future) Guide

This guide deploys the Streamlit app with Docker and Kubernetes, routes traffic from `chandudevopai.shop`, and keeps a future-ready Dhan API tab in the app.

1) Local Docker run

```bash
cd /opt/intratrade
vim deploy/docker/Dockerfile
vim deploy/docker/docker-compose.yml

# Build image
docker build -f deploy/docker/Dockerfile -t intratrade:latest .

# Run container
docker compose -f deploy/docker/docker-compose.yml up -d

# Verify
docker ps
docker logs intratrade_app --tail 100
```

Open `http://<server-ip>:8501`.

2) Kubernetes deployment (single node or cluster)

```bash
cd /opt/intratrade

# Create namespace + config
kubectl apply -f deploy/k8s/namespace.yaml
kubectl apply -f deploy/k8s/configmap.yaml

# Create secrets from template
cp deploy/k8s/secret.example.yaml deploy/k8s/secret.yaml
vim deploy/k8s/secret.yaml
```

```
kubectl apply -f deploy/k8s/secret.yaml

# Apply workload
kubectl apply -f deploy/k8s/deployment.yaml
kubectl apply -f deploy/k8s/service.yaml
kubectl apply -f deploy/k8s/ingress.yaml
kubectl apply -f deploy/k8s/hpa.yaml

# Verify
kubectl get all -n intratrade
kubectl get ingress -n intratrade
```

3) Push image to registry (recommended)

```bash
# Example with Docker Hub
docker tag intratrade:latest <dockerhub-user>/intratrade:latest
docker push <dockerhub-user>/intratrade:latest

# Then update deploy/k8s/deployment.yaml image:
# image: <dockerhub-user>/intratrade:latest
kubectl apply -f deploy/k8s/deployment.yaml
```

4) Domain routing

Set DNS A record:
- Host: `chandudevopai.shop`
- Value: your load balancer/public IP

If using NGINX ingress controller, ensure ports 80/443 are open.

For TLS (HTTPS):
- Install cert-manager in cluster.
- Add TLS secret and issuer in ingress manifest.

5) App feature included for future Dhan API

The app now includes strategy tab:
- `Dhan API (Future)`

Inside that tab:
- Auth panel for `client_id` and access token
- Market feed contract example

- Order router payload example
- Webhook event schema example
- Downloadable `dhan_order_template.csv`

Current status:
- Dhan tab is scaffold-only (safe for future extension)
- Existing bots continue to run normally

6) Useful operations

```bash
# Restart app deployment
kubectl rollout restart deployment intratrade-app -n intratrade

# Follow logs
kubectl logs -f deployment/intratrade-app -n intratrade

# Scale manually
kubectl scale deployment intratrade-app --replicas=2 -n intratrade

# Remove everything
kubectl delete ns intratrade
```

INTRATRADE COMBINED DOCUMENTATION

Scheduler Guide

F:\Trading\data\pdf_html\SCHEDULER_GUIDE.html

Scheduler Guide

Auto Backtest Scheduler Guide

This sets fully automatic backtest + paper logs without manual intervention.

Output files updated each run
- `data/live_ohlcv.csv`
- `data/backtest_results_all.csv`
- `data/backtest_results_history.csv`
- `data/paper_trading_logs_all.csv`

Windows Task Scheduler

1) Test manually once
```powershell
cd F:\Trading
powershell -NoProfile -ExecutionPolicy Bypass -File .\tools\run_auto_backtest.ps1
```

2) Register recurring task (every 15 minutes)
```powershell
cd F:\Trading
powershell -NoProfile -ExecutionPolicy Bypass -File .\deploy\scheduler\register_windows_task.ps1 -TaskName "IntratradeAutoBacktest" -Schedule MINUTE -Modifier 15 -StartTime "09:15"
```

3) Run on demand
```powershell
schtasks /Run /TN "IntratradeAutoBacktest"
```

4) Check status
```powershell
schtasks /Query /TN "IntratradeAutoBacktest" /V /FO LIST
```

5) Delete task
```powershell
```

```
schtasks /Delete /TN "IntratradeAutoBacktest" /F
```

Linux Cron (if deployed on Linux)

1) Make runner executable
```bash
cd /opt/intratrade
chmod +x deploy/scheduler/run_auto_backtest.sh
```

2) Add cron entry (every 15 minutes)
```bash
crontab -e
```
Add this line:
```cron
*/15 * * * * REPO_DIR=/opt/intratrade SYMBOL=RELIANCE.NS INTERVAL=5m PERIOD=5d
EXEC_SYMBOL=RELIANCE /opt/intratrade/deploy/scheduler/run_auto_backtest.sh >>
/opt/intratrade/data/cron_backtest.log 2>&1
```

3) Verify cron installed
```bash
crontab -l
```

Customize strategy feed
- Symbol: `RELIANCE.NS` / `^NSEI` / other Yahoo ticker
- Interval: `1m|2m|5m|15m|30m|60m|1d`
- Period: `1d|5d|1mo`

Notes
- Uses live OHLCV fetch from Yahoo on each run.
- No manual CSV upload needed.
- NIFTY quantity enforcement remains in execution engine (lot multiple logic).

INTRATRADE COMBINED DOCUMENTATION

Trading App Architecture

F:\Trading\data\docs\html\01_trading_app_architecture.html

01_trading_app_architecture

TRADING APP ARCHITECTURE DOCUMENTATION

Overview
This Trading App is designed with multiple tiers so market data, strategy logic, execution flow, notifications, and reporting stay organized and easy to manage.

WEB OR UI TIER
Usually UI tier is the one which has frontend and user interaction components.
In this project, UI tier is mainly Streamlit dashboard.
This tier is used for symbol selection, strategy selection, risk setup, lot size setup, execution mode, charts, signal review, and trade analysis.
If required in production, this UI can be exposed through Nginx reverse proxy.

APP OR STRATEGY TIER
App tier is the main business logic layer.
In this project it contains market data fetch, technical strategy execution, signal generation, strike selection, futures contract mapping, execution preparation, and reporting flow.
Major modules include Trading.py, live_ohlcv.py, breakout_bot.py, supply_demand.py, indicator_bot.py, one_trade_day.py, mtf_trade_bot.py, execution_engine.py, auto_backtest.py, and auto_run.py.
Usually this tier should not be directly open to the internet.

MARKET DATA TIER
Market data tier provides the raw inputs required for strategy calculations.
This project uses Yahoo Finance OHLCV data, NSE option chain data, NSE futures derivative data, and uploaded CSV files for testing or backtesting.
This tier is used for live candles, spot price, option metrics, futures contract data, and historical analysis.

EXECUTION TIER
Execution tier handles paper execution and live-execution-ready flow.
This tier converts trade signals into executable candidate rows, normalizes quantity using lot size, and writes execution logs.
Current project supports PAPER flow and LIVE-ready structure.
Future broker APIs can be attached here.

STORAGE TIER
Storage tier keeps the application outputs and logs.
This project mainly uses CSV, YAML, HTML, and PDF files.

Examples include live_ohlcv.csv, paper_trading_logs_all.csv, live_trading_logs_all.csv, backtest_results_all.csv, nifty_options_scored.csv, and nifty_futures_snapshot.csv.

NOTIFICATION TIER
Notification tier is used for communicating signals and summaries.
In this project Telegram is used for alerts, signal summaries, and report delivery.

REPORTING TIER
Reporting tier generates user-readable outputs.
This includes CSV reports, HTML summaries, PDF files, backtest summaries, and trade result snapshots.

COMMUNICATION FLOW
1. User opens the dashboard.
2. UI sends strategy settings to app tier.
3. App tier fetches market data from Yahoo or NSE sources.
4. Strategy modules generate BUY, SELL, or HOLD signals.
5. Contract layer converts signals into options or futures contracts.
6. Execution tier prepares paper or live-ready trade rows.
7. Storage tier writes logs and result files.
8. Notification tier sends Telegram alerts if enabled.
9. Reporting tier prepares summaries and downloadable documents.

DEPENDENCY FLOW
UI Tier depends on App Tier.
App Tier depends on Market Data Tier.
Execution Tier depends on App Tier.
Storage Tier depends on App Tier and Execution Tier.
Notification Tier depends on App Tier and Execution Tier.
Reporting Tier depends on Storage Tier and App Tier.

INTRATRADE COMBINED DOCUMENTATION

Trading App Deployment

F:\Trading\data\docs\html\02_trading_app_deployment.html

02_trading_app_deployment

TRADING APP DEPLOYMENT DOCUMENTATION

Overview
This document explains how the Trading App can be deployed for local use, server use, and production-style access.

LOCAL DEPLOYMENT
The project can run locally using Python and Streamlit.
Install dependencies from requirements.txt.
Launch the dashboard using Streamlit.
This mode is suitable for development, testing, and strategy review.

APPLICATION ENTRY POINTS
Main UI entry point is Trading.py.
Other entry points include auto_backtest.py, auto_run.py, nifty50.py, nifty_options.py, and nifty_futures.py.
These scripts support automation, scoring, and reporting outside the UI.

WEB ACCESS LAYER
For production-style deployment, Streamlit can be placed behind Nginx.
Nginx can work as reverse proxy for domain-based access.
This improves routing and gives a cleaner public endpoint.

SERVER RECOMMENDATION
Recommended server setup:
1. Linux or Windows VM.
2. Python runtime.
3. Streamlit service.
4. Optional Nginx reverse proxy.
5. Secure secrets storage for tokens and credentials.

CONFIGURATION REQUIREMENTS
Required configuration may include Telegram bot token, Telegram chat ID, broker credentials for future live trading, and optional AWS S3 configuration.
These values should be stored in environment variables, secrets, or protected config files.

DATA AND FILE PATHS
Application reads and writes under the data folder.

Make sure the deployment host has read and write permission for logs, report files, and snapshots.
If using persistent server deployment, this folder should be backed up or mounted on persistent storage.

SECURITY RECOMMENDATIONS
Only UI tier should be internet-facing.
App and execution internals should remain private.
Do not expose secrets inside source files.
Use HTTPS when publishing through a domain.
Protect broker APIs and notification credentials.

AUTOMATION OPTIONS
The project already supports scheduled or repeated runs using scripts and scheduler folders.
Auto backtests and reporting tasks can be scheduled using Windows Task Scheduler, cron, or container orchestration.

DEPLOYMENT MODES
Development Mode:
Run Streamlit locally for testing.

Internal Server Mode:
Run Streamlit on internal network for team use.

Production Mode:
Run Streamlit behind Nginx with restricted access, secrets management, and scheduled automation.

FUTURE EXTENSIONS
The deployment design allows addition of:
Broker API execution.
Container deployment.
Kubernetes deployment.
Cloud object storage for reports.
Monitoring and alerting.

INTRATRADE COMBINED DOCUMENTATION

Trading App Architecture Diagram

F:\Trading\data\docs\html\03_trading_app_architecture_diagram.html

03_trading_app_architecture_diagram

TRADING APP ARCHITECTURE DIAGRAM DOCUMENTATION

High Level Component View

```
USER
|
v
STREAMLIT DASHBOARD
|
v
APPLICATION LOGIC LAYER
|
+--> MARKET DATA CONNECTORS
| +--> Yahoo Finance OHLCV
| +--> NSE Option Chain
| +--> NSE Futures Data
|
+--> STRATEGY ENGINES
| +--> Breakout Strategy
| +--> Demand Supply Strategy
| +--> Indicator Strategy
| +--> One Trade Day Strategy
| +--> MTF 5m Strategy
|
+--> CONTRACT MAPPING LAYER
| +--> Option Strike Selection
| +--> Futures Contract Attachment
|
+--> EXECUTION ENGINE
| +--> Paper Execution
| +--> Live Ready Execution
|
+--> NOTIFICATION LAYER
| +--> Telegram Alerts
|
+--> REPORTING LAYER
+--> CSV Outputs
```

+--> HTML Reports
+--> PDF Reports

STORAGE LAYER
+--> CSV Files
+--> YAML Rules
+--> HTML Files
+--> PDF Files
+--> Optional S3 Upload

Communication Notes
The dashboard does not directly execute trades by itself.
It passes user-selected settings into the application logic layer.
The application logic layer fetches market data, runs strategies, maps contracts, and prepares execution rows.
The execution engine stores output into log files.
The reporting layer converts those outputs into readable summaries.
The notification layer shares important signals through Telegram.

Security Notes
UI can be exposed with controlled access.
Execution internals and secrets should remain protected.
Broker integrations should be attached only inside secure backend flow.

INTRATRADE COMBINED DOCUMENTATION

Trading App Production Runbook

F:\Trading\data\docs\html\04_trading_app_production_runbook.html

04_trading_app_production_runbook

TRADING APP PRODUCTION RUNBOOK

Purpose
This runbook explains how to operate, monitor, and troubleshoot the Trading App in a production or semi-production environment.

STARTUP CHECKLIST
1. Verify Python environment is available.
2. Verify required packages are installed.
3. Verify Streamlit app starts successfully.
4. Verify data folder is writable.
5. Verify Telegram credentials are configured if alerts are needed.
6. Verify broker credentials are configured if future live execution is enabled.

RUNTIME CHECKS
1. Confirm dashboard is reachable.
2. Confirm OHLCV data is loading.
3. Confirm strategy outputs are generated.
4. Confirm paper execution logs are updating.
5. Confirm report files are being created.
6. Confirm Telegram alerts are delivered when enabled.

DAILY OPERATIONS
Review live candles and trade signals.
Check paper or live trading logs.
Check backtest summary outputs.
Archive important CSV and PDF reports.
Validate that scheduled tasks completed without failure.

INCIDENT SCENARIOS
No market data:
Check external data provider access and symbol formatting.

No trade signals:
Check strategy settings, timeframe, and available candles.

Execution log not updating:

Check write permission on data directory and execution mode.

Telegram not sending:
Check token, chat ID, internet access, and notifier settings.

Wrong contract mapping:
Check whether Options or Futures mode is selected.
Check strike step, moneyness, and symbol normalization.

BACKUP RECOMMENDATION
Back up data folder regularly.
Critical files include paper trading logs, live trading logs, backtest summaries, and generated reports.

CHANGE MANAGEMENT
Apply changes first in local or testing environment.
Validate strategies and outputs.
Then promote to production server.
Avoid editing live configuration without backup.

FUTURE OPERATIONS IMPROVEMENT
Add service monitoring.
Add broker health checks.
Add log rotation.
Add alerting for failed scheduled runs.
Add persistent cloud storage for reports.

INTRATRADE COMBINED DOCUMENTATION

Client Executive Overview

F:\Trading\data\docs\client_facing\html\01_executive_overview.html

01_executive_overview

TRADING APP EXECUTIVE OVERVIEW

Overview
This Trading App is a unified trading workspace designed for market analysis, strategy execution, trade review, and reporting.
It brings live market data, multiple technical strategies, contract mapping, execution flow, and reporting into one operational platform.

Business Value
The application helps traders and operations teams reduce manual work by combining market monitoring, strategy testing, signal generation, contract preparation, and reporting in one place.

Core Capabilities
Live OHLCV market data viewing.
Multiple strategy engines.
Options contract selection.
Futures contract mode.
Paper execution workflow.
Live-execution-ready architecture.
Telegram alerting.
CSV, HTML, and PDF reporting.

User Benefits
Faster review of opportunities.
Structured trading workflow.
Centralized trade logs.
Repeatable backtesting and reporting.
Clear separation between analysis and execution.

Target Usage
This platform is suitable for active traders, internal trading desks, research teams, and semi-automated strategy operations.

INTRATRADE COMBINED DOCUMENTATION

Client Architecture

F:\Trading\data\docs\client_facing\html\02_client_architecture.html

02_client_architecture

TRADING APP CLIENT ARCHITECTURE

User Interface Layer
The application uses a Streamlit dashboard as the main user-facing layer.
Users can select symbols, intervals, strategies, risk settings, execution mode, and contract type from a single interface.

Application Layer
The backend logic is implemented in Python modules that manage market data processing, strategy execution, signal preparation, contract conversion, and trade output generation.

Market Data Layer
The system consumes Yahoo Finance OHLCV data, NSE option-chain data, NSE futures data, and uploaded CSV datasets.

Execution Layer
The application currently supports paper execution and is structured for future broker API integration.

Reporting Layer
The system generates CSV outputs, HTML summaries, PDF documents, and Telegram notifications.

Storage Layer
Operational files are stored in structured folders so logs, snapshots, reports, and rule files remain accessible and auditable.

INTRATRADE COMBINED DOCUMENTATION

Client Deployment

F:\Trading\data\docs\client_facing\html\03_client_deployment.html

03_client_deployment

TRADING APP CLIENT DEPLOYMENT SUMMARY

Deployment Style
The platform can run locally for development, on an internal server for team usage, or behind Nginx for controlled production-style access.

Recommended Stack
Python runtime.
Streamlit application server.
Optional Nginx reverse proxy.
Persistent storage for logs and reports.
Secure secret storage for tokens and credentials.

Operational Readiness
The design already supports scheduled jobs, repeatable backtests, report generation, and phased extension toward broker integrations.

Security Guidance
Only the user interface should be internet-facing.
Execution internals, credentials, and broker integrations should remain protected.

INTRATRADE COMBINED DOCUMENTATION

Broker Integration Flow

F:\Trading\data\docs\client_facing\html\04_broker_integration_flow.html

04_broker_integration_flow

TRADING APP BROKER INTEGRATION FLOW

Current State
The platform already supports signal generation, quantity preparation, and execution logging.
Paper trading is active and live integration is prepared as the next extension point.

Future Broker Flow
1. Strategy generates BUY or SELL signal.
2. Contract layer maps the signal to options or futures.
3. Execution engine prepares order payload.
4. Broker adapter validates symbol, quantity, and product type.
5. Order is sent to broker API.
6. Response is stored in execution log.
7. Telegram notification is sent if enabled.

Integration Targets
Dhan API.
Zerodha API.
Angel One API.
Other REST or WebSocket brokers.

Operational Requirement
Broker APIs should be attached only after stable paper-trading validation and proper secret management.

INTRATRADE COMBINED DOCUMENTATION

Intratrade Master Documentation

F:\Trading\data\docs\branded\intratrade_master_documentation.html

INTRATRADE

Trading App Master Documentation

Architecture, deployment, diagram summary, and production runbook in one document.

1. Architecture

TRADING APP ARCHITECTURE DOCUMENTATION

Overview
This Trading App is designed with multiple tiers so market data, strategy logic, execution flow, notifications, and reporting stay organized and easy to manage.

WEB OR UI TIER
Usually UI tier is the one which has frontend and user interaction components.
In this project, UI tier is mainly Streamlit dashboard.
This tier is used for symbol selection, strategy selection, risk setup, lot size setup, execution mode, charts, signal review, and trade analysis.
If required in production, this UI can be exposed through Nginx reverse proxy.

APP OR STRATEGY TIER
App tier is the main business logic layer.
In this project it contains market data fetch, technical strategy execution, signal generation, strike selection, futures contract mapping, execution preparation, and reporting flow.
Major modules include Trading.py, live_ohlcv.py, breakout_bot.py, supply_demand.py, indicator_bot.py, one_trade_day.py, mtf_trade_bot.py, execution_engine.py, auto_backtest.py, and auto_run.py.
Usually this tier should not be directly open to the internet.

MARKET DATA TIER
Market data tier provides the raw inputs required for strategy calculations.
This project uses Yahoo Finance OHLCV data, NSE option chain data, NSE futures derivative data, and uploaded CSV files for testing or backtesting.
This tier is used for live candles, spot price, option metrics, futures contract data, and historical analysis.

EXECUTION TIER
Execution tier handles paper execution and live-execution-ready flow.
This tier converts trade signals into executable candidate rows, normalizes quantity using lot size, and writes execution logs.
Current project supports PAPER flow and LIVE-ready structure.
Future broker APIs can be attached here.

STORAGE TIER
Storage tier keeps the application outputs and logs.
This project mainly uses CSV, YAML, HTML, and PDF files.
Examples include live_ohlcv.csv, paper_trading_logs_all.csv, live_trading_logs_all.csv, backtest_results_all.csv,

nifty_options_scored.csv, and nifty_futures_snapshot.csv.

NOTIFICATION TIER
Notification tier is used for communicating signals and summaries.
In this project Telegram is used for alerts, signal summaries, and report delivery.

REPORTING TIER
Reporting tier generates user-readable outputs.
This includes CSV reports, HTML summaries, PDF files, backtest summaries, and trade result snapshots.

COMMUNICATION FLOW
1. User opens the dashboard.
2. UI sends strategy settings to app tier.
3. App tier fetches market data from Yahoo or NSE sources.
4. Strategy modules generate BUY, SELL, or HOLD signals.
5. Contract layer converts signals into options or futures contracts.
6. Execution tier prepares paper or live-ready trade rows.
7. Storage tier writes logs and result files.
8. Notification tier sends Telegram alerts if enabled.
9. Reporting tier prepares summaries and downloadable documents.

DEPENDENCY FLOW
UI Tier depends on App Tier.
App Tier depends on Market Data Tier.
Execution Tier depends on App Tier.
Storage Tier depends on App Tier and Execution Tier.
Notification Tier depends on App Tier and Execution Tier.
Reporting Tier depends on Storage Tier and App Tier.

2. Deployment

TRADING APP DEPLOYMENT DOCUMENTATION

Overview
This document explains how the Trading App can be deployed for local use, server use, and production-style access.

LOCAL DEPLOYMENT
The project can run locally using Python and Streamlit.
Install dependencies from requirements.txt.
Launch the dashboard using Streamlit.
This mode is suitable for development, testing, and strategy review.

APPLICATION ENTRY POINTS
Main UI entry point is Trading.py.

Other entry points include auto_backtest.py, auto_run.py, nifty50.py, nifty_options.py, and nifty_futures.py.
These scripts support automation, scoring, and reporting outside the UI.

WEB ACCESS LAYER
For production-style deployment, Streamlit can be placed behind Nginx.
Nginx can work as reverse proxy for domain-based access.
This improves routing and gives a cleaner public endpoint.

SERVER RECOMMENDATION
Recommended server setup:
1. Linux or Windows VM.
2. Python runtime.
3. Streamlit service.
4. Optional Nginx reverse proxy.
5. Secure secrets storage for tokens and credentials.

CONFIGURATION REQUIREMENTS
Required configuration may include Telegram bot token, Telegram chat ID, broker credentials for future live trading, and optional AWS S3 configuration.
These values should be stored in environment variables, secrets, or protected config files.

DATA AND FILE PATHS
Application reads and writes under the data folder.
Make sure the deployment host has read and write permission for logs, report files, and snapshots.
If using persistent server deployment, this folder should be backed up or mounted on persistent storage.

SECURITY RECOMMENDATIONS
Only UI tier should be internet-facing.
App and execution internals should remain private.
Do not expose secrets inside source files.
Use HTTPS when publishing through a domain.
Protect broker APIs and notification credentials.

AUTOMATION OPTIONS
The project already supports scheduled or repeated runs using scripts and scheduler folders.
Auto backtests and reporting tasks can be scheduled using Windows Task Scheduler, cron, or container orchestration.

DEPLOYMENT MODES
Development Mode:
Run Streamlit locally for testing.

Internal Server Mode:
Run Streamlit on internal network for team use.

Production Mode:
Run Streamlit behind Nginx with restricted access, secrets management, and scheduled automation.

FUTURE EXTENSIONS
The deployment design allows addition of:
Broker API execution.
Container deployment.
Kubernetes deployment.
Cloud object storage for reports.
Monitoring and alerting.

3. Architecture Diagram Summary

TRADING APP ARCHITECTURE DIAGRAM DOCUMENTATION

High Level Component View

```
USER
|
v
STREAMLIT DASHBOARD
|
v
APPLICATION LOGIC LAYER
|
+--> MARKET DATA CONNECTORS
| +--> Yahoo Finance OHLCV
| +--> NSE Option Chain
| +--> NSE Futures Data
|
+--> STRATEGY ENGINES
| +--> Breakout Strategy
| +--> Demand Supply Strategy
| +--> Indicator Strategy
| +--> One Trade Day Strategy
| +--> MTF 5m Strategy
|
+--> CONTRACT MAPPING LAYER
| +--> Option Strike Selection
| +--> Futures Contract Attachment
|
+--> EXECUTION ENGINE
| +--> Paper Execution
| +--> Live Ready Execution
|
```

```
+--> NOTIFICATION LAYER
| +--> Telegram Alerts
|
+--> REPORTING LAYER
+--> CSV Outputs
+--> HTML Reports
+--> PDF Reports

STORAGE LAYER
+--> CSV Files
+--> YAML Rules
+--> HTML Files
+--> PDF Files
+--> Optional S3 Upload
```

Communication Notes
The dashboard does not directly execute trades by itself.
It passes user-selected settings into the application logic layer.
The application logic layer fetches market data, runs strategies, maps contracts, and prepares execution rows.
The execution engine stores output into log files.
The reporting layer converts those outputs into readable summaries.
The notification layer shares important signals through Telegram.

Security Notes
UI can be exposed with controlled access.
Execution internals and secrets should remain protected.
Broker integrations should be attached only inside secure backend flow.

4. Production Runbook

TRADING APP PRODUCTION RUNBOOK

Purpose
This runbook explains how to operate, monitor, and troubleshoot the Trading App in a production or semi-production environment.

STARTUP CHECKLIST
1. Verify Python environment is available.
2. Verify required packages are installed.
3. Verify Streamlit app starts successfully.
4. Verify data folder is writable.
5. Verify Telegram credentials are configured if alerts are needed.
6. Verify broker credentials are configured if future live execution is enabled.

RUNTIME CHECKS

1. Confirm dashboard is reachable.
2. Confirm OHLCV data is loading.
3. Confirm strategy outputs are generated.
4. Confirm paper execution logs are updating.
5. Confirm report files are being created.
6. Confirm Telegram alerts are delivered when enabled.

DAILY OPERATIONS
Review live candles and trade signals.
Check paper or live trading logs.
Check backtest summary outputs.
Archive important CSV and PDF reports.
Validate that scheduled tasks completed without failure.

INCIDENT SCENARIOS
No market data:
Check external data provider access and symbol formatting.

No trade signals:
Check strategy settings, timeframe, and available candles.

Execution log not updating:
Check write permission on data directory and execution mode.

Telegram not sending:
Check token, chat ID, internet access, and notifier settings.

Wrong contract mapping:
Check whether Options or Futures mode is selected.
Check strike step, moneyness, and symbol normalization.

BACKUP RECOMMENDATION
Back up data folder regularly.
Critical files include paper trading logs, live trading logs, backtest summaries, and generated reports.

CHANGE MANAGEMENT
Apply changes first in local or testing environment.
Validate strategies and outputs.
Then promote to production server.
Avoid editing live configuration without backup.

FUTURE OPERATIONS IMPROVEMENT
Add service monitoring.
Add broker health checks.
Add log rotation.

Add alerting for failed scheduled runs.

Add persistent cloud storage for reports.

INTRATRADE COMBINED DOCUMENTATION

Intratrade Client Proposal

F:\Trading\data\docs\branded\intratrade_client_proposal.html

INTRATRADE

Trading App Client Proposal

Platform overview, architecture direction, deployment summary, and broker integration roadmap.

Executive Overview

TRADING APP EXECUTIVE OVERVIEW

Overview
This Trading App is a unified trading workspace designed for market analysis, strategy execution, trade review, and reporting.
It brings live market data, multiple technical strategies, contract mapping, execution flow, and reporting into one operational platform.

Business Value
The application helps traders and operations teams reduce manual work by combining market monitoring, strategy testing, signal generation, contract preparation, and reporting in one place.

Core Capabilities
Live OHLCV market data viewing.
Multiple strategy engines.
Options contract selection.
Futures contract mode.
Paper execution workflow.
Live-execution-ready architecture.
Telegram alerting.
CSV, HTML, and PDF reporting.

User Benefits
Faster review of opportunities.
Structured trading workflow.
Centralized trade logs.
Repeatable backtesting and reporting.
Clear separation between analysis and execution.

Target Usage
This platform is suitable for active traders, internal trading desks, research teams, and semi-automated strategy operations.

Architecture Summary

TRADING APP CLIENT ARCHITECTURE

User Interface Layer

The application uses a Streamlit dashboard as the main user-facing layer.
Users can select symbols, intervals, strategies, risk settings, execution mode, and contract type from a single interface.

Application Layer
The backend logic is implemented in Python modules that manage market data processing, strategy execution, signal preparation, contract conversion, and trade output generation.

Market Data Layer
The system consumes Yahoo Finance OHLCV data, NSE option-chain data, NSE futures data, and uploaded CSV datasets.

Execution Layer
The application currently supports paper execution and is structured for future broker API integration.

Reporting Layer
The system generates CSV outputs, HTML summaries, PDF documents, and Telegram notifications.

Storage Layer
Operational files are stored in structured folders so logs, snapshots, reports, and rule files remain accessible and auditable.

Deployment Summary

TRADING APP CLIENT DEPLOYMENT SUMMARY

Deployment Style
The platform can run locally for development, on an internal server for team usage, or behind Nginx for controlled production-style access.

Recommended Stack
Python runtime.
Streamlit application server.
Optional Nginx reverse proxy.
Persistent storage for logs and reports.
Secure secret storage for tokens and credentials.

Operational Readiness
The design already supports scheduled jobs, repeatable backtests, report generation, and phased extension toward broker integrations.

Security Guidance
Only the user interface should be internet-facing.
Execution internals, credentials, and broker integrations should remain protected.

Broker Integration Roadmap

TRADING APP BROKER INTEGRATION FLOW

Current State
The platform already supports signal generation, quantity preparation, and execution logging.
Paper trading is active and live integration is prepared as the next extension point.

Future Broker Flow
1. Strategy generates BUY or SELL signal.
2. Contract layer maps the signal to options or futures.
3. Execution engine prepares order payload.
4. Broker adapter validates symbol, quantity, and product type.
5. Order is sent to broker API.
6. Response is stored in execution log.
7. Telegram notification is sent if enabled.

Integration Targets
Dhan API.
Zerodha API.
Angel One API.
Other REST or WebSocket brokers.

Operational Requirement
Broker APIs should be attached only after stable paper-trading validation and proper secret management.

INTRATRADE COMBINED DOCUMENTATION

Intratrade Commercial Proposal

F:\Trading\data\docs\branded\intratrade_commercial_proposal.html

INTRATRADE

Trading App Commercial Proposal

Proposal for a production-ready trading platform that combines live market data, multi-strategy execution workflow, options and futures contract mapping, reporting, and future broker API integration.

Client Proposal + Scope + Commercial Summary

Executive Summary

This proposal presents a structured trading application designed to support live market monitoring, strategy-based trade generation, paper execution, reporting, and future broker connectivity. The platform is suitable for internal desks, trading operations teams, and semi-automated research workflows.

Solution Overview

Frontend Layer
Streamlit dashboard for strategy controls, live charts, execution review, and reporting.
Application Layer
Python-based strategy engine, signal flow, futures/options mapping, execution engine, and reporting logic.
Data Layer
Yahoo OHLCV, NSE option-chain inputs, NSE futures data, CSV backtest files, and rule-based scoring.
Operations Layer
Paper execution logs, Telegram alerts, PDF/CSV reports, and future broker API adapters.

Scope of Work

Included Scope

- Trading dashboard for live analysis and strategy control
- Support for breakout, demand-supply, indicator, one-trade-day, and MTF strategies
- Options contract selection and futures contract mode
- Paper trading workflow and execution logs
- Telegram alert integration
- CSV, HTML, and PDF reporting outputs
- Deployment-ready documentation and operational runbook

Optional Add-ons

- Broker API live order integration
- Cloud deployment and monitoring
- Authentication and role-based access
- Database-backed execution history
- Advanced analytics and trader performance dashboards

Implementation Phases

Phase	Focus	Outcome
Phase 1	Core platform	Dashboard, strategies, paper execution, reporting
Phase 2	Production hardening	Deployment setup, security controls, scheduling, backups
Phase 3	Broker integration	Live order routing, validation, execution audit trail
Phase 4	Scale and analytics	Monitoring, persistence, multi-user operations, advanced insights

Commercial Summary

Starter Implementation
Custom Quote
Recommended when scope is finalized after business discussion.
Suggested Pricing Model
Phase-based delivery with milestone approvals for architecture, deployment readiness, and live integration.

Indicative Commercial Options

Package	Description	Commercial Style
Foundation	Dashboard, strategies, paper trading, reports	Fixed milestone pricing
Professional	Foundation plus production deployment and operations	Fixed milestone pricing
Enterprise	Professional plus broker integration and monitoring	Custom engagement pricing

Assumptions

- Live broker integration is handled as a later phase after paper validation
- Market data provider access remains available from approved sources
- Client branding assets can be added once logo and identity files are shared
- Final commercials will depend on deployment environment and broker requirements

Prepared as a branded proposal draft for Intratrade. Logo treatment currently uses text branding and can be replaced with your official logo anytime.

INTRATRADE COMBINED DOCUMENTATION

Intratrade Premium Master Dark

F:\Trading\data\docs\branded\intratrade_premium_master_dark.html

INTRATRADE

Premium Technical Documentation

Master reference for architecture, deployment flow, diagram notes, and production operations.

Architecture

TRADING APP ARCHITECTURE DOCUMENTATION

Overview
This Trading App is designed with multiple tiers so market data, strategy logic, execution flow, notifications, and reporting stay organized and easy to manage.

WEB OR UI TIER
Usually UI tier is the one which has frontend and user interaction components.
In this project, UI tier is mainly Streamlit dashboard.
This tier is used for symbol selection, strategy selection, risk setup, lot size setup, execution mode, charts, signal review, and trade analysis.
If required in production, this UI can be exposed through Nginx reverse proxy.

APP OR STRATEGY TIER
App tier is the main business logic layer.
In this project it contains market data fetch, technical strategy execution, signal generation, strike selection, futures contract mapping, execution preparation, and reporting flow.
Major modules include Trading.py, live_ohlcv.py, breakout_bot.py, supply_demand.py, indicator_bot.py, one_trade_day.py, mtf_trade_bot.py, execution_engine.py, auto_backtest.py, and auto_run.py.
Usually this tier should not be directly open to the internet.

MARKET DATA TIER
Market data tier provides the raw inputs required for strategy calculations.
This project uses Yahoo Finance OHLCV data, NSE option chain data, NSE futures derivative data, and uploaded CSV files for testing or backtesting.
This tier is used for live candles, spot price, option metrics, futures contract data, and historical analysis.

EXECUTION TIER
Execution tier handles paper execution and live-execution-ready flow.
This tier converts trade signals into executable candidate rows, normalizes quantity using lot size, and writes execution logs.

Current project supports PAPER flow and LIVE-ready structure.
Future broker APIs can be attached here.

STORAGE TIER
Storage tier keeps the application outputs and logs.
This project mainly uses CSV, YAML, HTML, and PDF files.
Examples include live_ohlcv.csv, paper_trading_logs_all.csv, live_trading_logs_all.csv, backtest_results_all.csv, nifty_options_scored.csv, and nifty_futures_snapshot.csv.

NOTIFICATION TIER
Notification tier is used for communicating signals and summaries.
In this project Telegram is used for alerts, signal summaries, and report delivery.

REPORTING TIER
Reporting tier generates user-readable outputs.
This includes CSV reports, HTML summaries, PDF files, backtest summaries, and trade result snapshots.

COMMUNICATION FLOW
1. User opens the dashboard.
2. UI sends strategy settings to app tier.
3. App tier fetches market data from Yahoo or NSE sources.
4. Strategy modules generate BUY, SELL, or HOLD signals.
5. Contract layer converts signals into options or futures contracts.
6. Execution tier prepares paper or live-ready trade rows.
7. Storage tier writes logs and result files.
8. Notification tier sends Telegram alerts if enabled.
9. Reporting tier prepares summaries and downloadable documents.

DEPENDENCY FLOW
UI Tier depends on App Tier.
App Tier depends on Market Data Tier.
Execution Tier depends on App Tier.
Storage Tier depends on App Tier and Execution Tier.
Notification Tier depends on App Tier and Execution Tier.
Reporting Tier depends on Storage Tier and App Tier.

Deployment

TRADING APP DEPLOYMENT DOCUMENTATION

Overview
This document explains how the Trading App can be deployed for local use, server use, and production-style access.

LOCAL DEPLOYMENT

The project can run locally using Python and Streamlit.
Install dependencies from requirements.txt.
Launch the dashboard using Streamlit.
This mode is suitable for development, testing, and strategy review.

APPLICATION ENTRY POINTS
Main UI entry point is Trading.py.
Other entry points include auto_backtest.py, auto_run.py, nifty50.py, nifty_options.py, and nifty_futures.py.
These scripts support automation, scoring, and reporting outside the UI.

WEB ACCESS LAYER
For production-style deployment, Streamlit can be placed behind Nginx.
Nginx can work as reverse proxy for domain-based access.
This improves routing and gives a cleaner public endpoint.

SERVER RECOMMENDATION
Recommended server setup:
1. Linux or Windows VM.
2. Python runtime.
3. Streamlit service.
4. Optional Nginx reverse proxy.
5. Secure secrets storage for tokens and credentials.

CONFIGURATION REQUIREMENTS
Required configuration may include Telegram bot token, Telegram chat ID, broker credentials for future live trading, and optional AWS S3 configuration.
These values should be stored in environment variables, secrets, or protected config files.

DATA AND FILE PATHS
Application reads and writes under the data folder.
Make sure the deployment host has read and write permission for logs, report files, and snapshots.
If using persistent server deployment, this folder should be backed up or mounted on persistent storage.

SECURITY RECOMMENDATIONS
Only UI tier should be internet-facing.
App and execution internals should remain private.
Do not expose secrets inside source files.
Use HTTPS when publishing through a domain.
Protect broker APIs and notification credentials.

AUTOMATION OPTIONS
The project already supports scheduled or repeated runs using scripts and scheduler folders.
Auto backtests and reporting tasks can be scheduled using Windows Task Scheduler, cron, or container orchestration.

DEPLOYMENT MODES
Development Mode:
Run Streamlit locally for testing.

Internal Server Mode:
Run Streamlit on internal network for team use.

Production Mode:
Run Streamlit behind Nginx with restricted access, secrets management, and scheduled automation.

FUTURE EXTENSIONS
The deployment design allows addition of:
Broker API execution.
Container deployment.
Kubernetes deployment.
Cloud object storage for reports.
Monitoring and alerting.

Diagram Summary

TRADING APP ARCHITECTURE DIAGRAM DOCUMENTATION

High Level Component View

```
USER
|
v
STREAMLIT DASHBOARD
|
v
APPLICATION LOGIC LAYER
|
+--> MARKET DATA CONNECTORS
| +--> Yahoo Finance OHLCV
| +--> NSE Option Chain
| +--> NSE Futures Data
|
+--> STRATEGY ENGINES
| +--> Breakout Strategy
| +--> Demand Supply Strategy
| +--> Indicator Strategy
| +--> One Trade Day Strategy
| +--> MTF 5m Strategy
|
+--> CONTRACT MAPPING LAYER
```

```
| +--> Option Strike Selection
| +--> Futures Contract Attachment
|
+--> EXECUTION ENGINE
| +--> Paper Execution
| +--> Live Ready Execution
|
+--> NOTIFICATION LAYER
| +--> Telegram Alerts
|
+--> REPORTING LAYER
+--> CSV Outputs
+--> HTML Reports
+--> PDF Reports

STORAGE LAYER
+--> CSV Files
+--> YAML Rules
+--> HTML Files
+--> PDF Files
+--> Optional S3 Upload
```

Communication Notes
The dashboard does not directly execute trades by itself.
It passes user-selected settings into the application logic layer.
The application logic layer fetches market data, runs strategies, maps contracts, and prepares execution rows.
The execution engine stores output into log files.
The reporting layer converts those outputs into readable summaries.
The notification layer shares important signals through Telegram.

Security Notes
UI can be exposed with controlled access.
Execution internals and secrets should remain protected.
Broker integrations should be attached only inside secure backend flow.

Runbook

TRADING APP PRODUCTION RUNBOOK

Purpose
This runbook explains how to operate, monitor, and troubleshoot the Trading App in a production or semi-production environment.

STARTUP CHECKLIST
1. Verify Python environment is available.

2. Verify required packages are installed.
3. Verify Streamlit app starts successfully.
4. Verify data folder is writable.
5. Verify Telegram credentials are configured if alerts are needed.
6. Verify broker credentials are configured if future live execution is enabled.

RUNTIME CHECKS
1. Confirm dashboard is reachable.
2. Confirm OHLCV data is loading.
3. Confirm strategy outputs are generated.
4. Confirm paper execution logs are updating.
5. Confirm report files are being created.
6. Confirm Telegram alerts are delivered when enabled.

DAILY OPERATIONS
Review live candles and trade signals.
Check paper or live trading logs.
Check backtest summary outputs.
Archive important CSV and PDF reports.
Validate that scheduled tasks completed without failure.

INCIDENT SCENARIOS
No market data:
Check external data provider access and symbol formatting.

No trade signals:
Check strategy settings, timeframe, and available candles.

Execution log not updating:
Check write permission on data directory and execution mode.

Telegram not sending:
Check token, chat ID, internet access, and notifier settings.

Wrong contract mapping:
Check whether Options or Futures mode is selected.
Check strike step, moneyness, and symbol normalization.

BACKUP RECOMMENDATION
Back up data folder regularly.
Critical files include paper trading logs, live trading logs, backtest summaries, and generated reports.

CHANGE MANAGEMENT
Apply changes first in local or testing environment.
Validate strategies and outputs.

Then promote to production server.
Avoid editing live configuration without backup.

FUTURE OPERATIONS IMPROVEMENT
Add service monitoring.
Add broker health checks.
Add log rotation.
Add alerting for failed scheduled runs.
Add persistent cloud storage for reports.